

OpenMP offloading を用いた並列プログラムの異種 GPU 上での評価

東京科学大学 情報理工学院 数理・計算科学系
21B30971 吉田 和真
指導教員 遠藤 敏夫

1 はじめに

計算の高速化の手段として, CPU や GPU の並列化手法が挙げられる. CPU 並列化手法としては OpenMP, GPU 並列化手法としては CUDA や OpenACC, OpenCL が有名であるが, 本研究では AMD GPU と NVIDIA GPU 上で使用可能な OpenMP offloading[1] という GPU 並列化手法を取り上げる.

2 実験用プログラム

実験は行列積演算プログラム, diffusion プログラム, Rodinia ベンチマーク [2] の hotspot プログラムの 3 つを並列化して行った. 以下, 行列積演算プログラムと diffusion プログラムを紹介する.

ソースコード 1: 行列積演算プログラム

```
1 int matmul()
2 {
3     int i, j, l;
4     for (j = 0; j < n; j++) {
5         // 以下のループの順序は並列化手法に
6         // よって入れ替える
7         for (l = 0; l < k; l++) {
8             double blj = B[l+j*k];
9             for (i = 0; i < m; i++) {
10                double ail = A[i+l*m];
11                C[i+j*m] += ail*blj;
12            }
13        }
14    }
15 }
```

ソースコード 2: diffusion プログラム

```
1 void calc(long nx, long ny, long nt)
2 {
3     long t, x, y;
4     for (t = 0; t < nt; t++) {
5         long from = t%2;
6         long to = (t+1)%2;
7         for (y = 1; y < ny-1; y++) {
8             for (x = 1; x < nx-1; x++) {
9                 data[IDX(to,x,y)] = 0.2 * (
10                    data[IDX(from,x,y)] + data[
11                        IDX(from,x-1,y)] + data[
12                            IDX(from,x+1,y)] + data[
13                                IDX(from,x,y-1)] + data[
14                                    IDX(from,x,y+1)]);
15            }
16        }
17    }
18 }
```

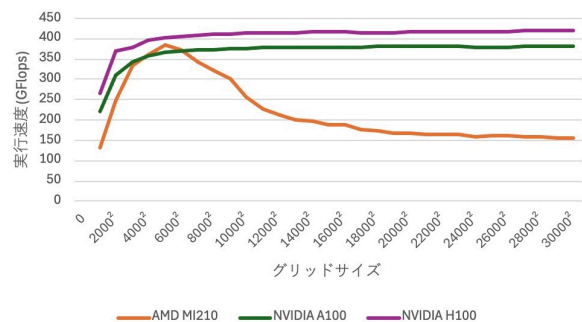


図 1: 各 GPU による diffusion プログラムの実行速度

3 実験と評価

行列積演算, diffusion を OpenMP offloading で実装し, サイズを増やしながら実験したところ, diffusion プログラムでのみ AMD GPU による実行速度の低下が見られた. 実験結果を図 1 に示す.

また, 同様のプログラムを OpenMP, OpenACC で実装し, 比較実験を行った. diffusion プログラムでは, OpenMP offloading は全体として, OpenACC の 0.7~0.8 倍の実行速度になることが分かった. また, OpenMP は OpenMP offloading や OpenACC より実行速度が劣る結果となった. NVIDIA A100 が搭載されたマシンでの実験結果を図 2 に示す.

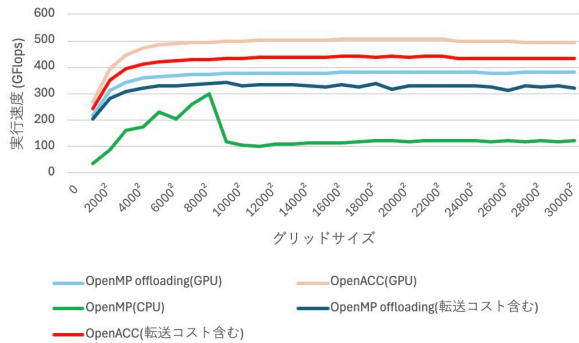


図 2: 各並列化手法における diffusion 実行速度

最後に行列積演算の 2 番目と 3 番目のループの順序を入れ替えて, CPU と GPU の性能ポータビリティに関する実験を行った. OpenMP と OpenMP offloading を用いた場合の実験結果を図 3, 4 に示す. これらの結果から, CPU 実行の OpenMP では j, l, i ループの方が, GPU 実行の OpenMP offloading では j, i, l ループの方が実行速度が速いことが分かった.

4 まとめと今後の課題

サイズの大きい計算では CPU 並列化手法より GPU 並列化手法の方が実行速度が速いことが分かった. また, OpenMP offloading は OpenACC より実行速度が遅くなってしまう場合があることも確認された. さらに, 行列積演算では CPU, GPU 間で性能ポータビリティが維持されず, それぞれの並列化手法で適切なループの順番があることが分かった. 今後の研究課題とし

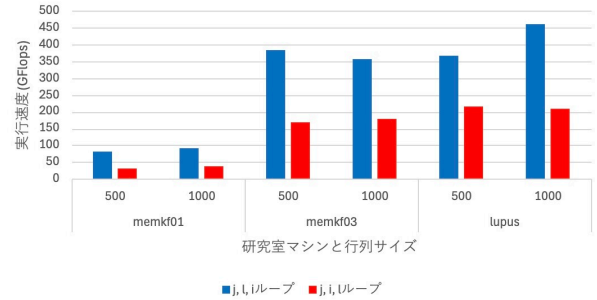


図 3: 異なる for ループでの OpenMP 実行速度

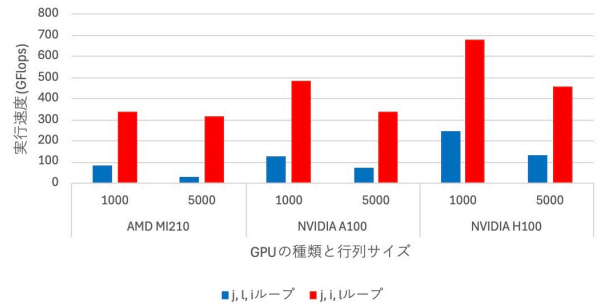


図 4: 異なる for ループでの OpenMP offloading 実行速度

ては, OpenMP offloading を用いた際に AMD GPU 上で実行速度が遅くなってしまったケースについて, 原因がキャッシュミスにあるのか, NVIDIA GPU との構造上の違いにあるのかを確かめることである. また, 他にも異なるデバイス上で性能ポータビリティが保たれないケースを見つけ, 原因の検証も行いたい.

参考文献

- [1] 河合 直聡, 三木 洋平, 星野 哲也, 塙 敏博, 中島 研吾. Openmp を用いた gpu オフローディングの有効性の評価. 情報処理学会研究報告, Vol. 2022-HPC-183, No. No.18, 2022.
- [2] OpenBenchmarking.
<https://openbenchmarking.org/test/pts/rodinia>.